

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

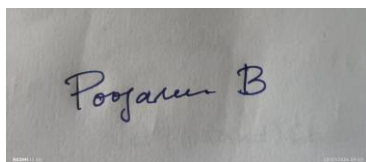
Weightage: 20%

QUESTIONS:

Total Marks: 20

Code of Conduct:

I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

A photograph of a piece of paper with a handwritten signature in blue ink. The signature appears to be 'Poojasree B'.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- Identify key components required in a navigation bar for an e-commerce website.
- Design a responsive navigation bar using appropriate CSS techniques.
- Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
- Evaluate the usability and suggest improvements for better user experience.

(a) Key components required in an e-commerce navigation bar

A responsive e-commerce navigation bar should include:

- **Company Logo** – Brand identity and home page link.
- **Navigation Links** – Home, Products, Categories, About, Contact.
- **Search Bar** – Quick product search.
- **Shopping Cart Icon** – Displays selected items.
- **User Account/Login** – Login, Register, Profile.
- **Wishlist (optional)** – Save favorite products.
- **Hamburger Menu** – Displays navigation items on mobile devices.

(b) Responsive Navigation Bar using HTML & CSS

HTML

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Navigation Bar</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<nav class="navbar">
  <div class="logo">ShopEasy</div>
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">Products</a></li>
    <li><a href="#">Categories</a></li>
    <li><a href="#">Contact</a></li>
    <li><a href="#">Cart</a></li>
  </ul>
  <div class="menu">&#9776;</div>
</nav>
</body>
</html>
```

CSS

```
*{
margin:0;
```

```
padding:0;
box-sizing:border-box;
}
```

```
body{
font-family:Arial;
}
```

```
.logo{
color:white;
font-size:24px;
font-weight:bold;
}
```

```
.nav-links{
display:flex;
list-style:none;
}
```

```
.nav-links li{
margin-left:20px;
}
```

```
.nav-links a{
color:white;
text-decoration:none;
}
```

```
.menu{
display:none;
color:white;
font-size:30px;
cursor:pointer;
}
```

```
@media(max-width:768px){
.nav-links{
display:none;
flex-direction:column;
width:100%;
background:#333;
}
```

```
.menu{
display:block;
}
}
```

(c) Navigation adaptation for smaller screens (Hamburger Menu)

On screens **768px or smaller**:

- Navigation links are hidden using `display: none`.
- The **hamburger icon (☰)** becomes visible.
- When clicked (using JavaScript), the menu expands vertically.

Example JavaScript:

```
<script>
const menu=document.querySelector(".menu");
const nav=document.querySelector(".nav-links");
menu.onclick=function(){
  nav.classList.toggle("show");
}
</script>
```

Add to CSS:

```
.show{
  display:flex;
}
```

(d) Usability Evaluation and Improvements

Evaluation

- Responsive layout works well on desktop, tablet, and mobile.
- Navigation remains clean and organized.
- Hamburger menu saves screen space on small devices.

Suggested Improvements

- Add smooth slide-down animation for the mobile menu.
- Highlight the active navigation link.
- Increase button size for easier touch interaction.
- Include search and cart icons.
- Improve accessibility using ARIA labels and keyboard navigation.
- Use sticky navigation so the menu remains visible while scrolling.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
- (b) Suggest appropriate **CSS animation or transition** for product interaction.
- (c) Design a hover effect for a product card.
- (d) Evaluate performance and user experience considerations.

(a) Suitable CSS properties for hover effects

Common CSS properties used for product hover effects are:

- transform – Scale, rotate, or move the product.
- transition – Creates smooth animation.
- box-shadow – Adds shadow for depth.
- opacity – Changes transparency.
- background-color – Changes card background.
- color – Changes text color.

(b) Appropriate CSS animation/transition

Use the transition property for smooth effects.

Example:

transition: transform 0.3s ease, box-shadow 0.3s ease;

Effects:

- Product slightly enlarges.
- Shadow becomes darker.
- Animation is smooth and responsive.

(c) Product Card Hover Effect

HTML

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="product-card">
  
  <h3>Wireless Headphones</h3>
  <p>$99</p>
  <button>Buy Now</button>
```

```
</div>
</body>
</html>
```

CSS

```
.product-card{
  width:250px;
  padding:15px;
  border:1px solid #ccc;
  border-radius:10px;
  text-align:center;
  transition:transform 0.3s ease, box-shadow 0.3s ease;
}
.product-card img{
  width:100%;
}
.product-card:hover{
  transform:scale(1.05);
  box-shadow:0 8px 20px rgba(0,0,0,0.3);
}

button{
  background:#007BFF;
  color:white;
  border:none;
  padding:10px 20px;
  cursor:pointer;
}

button:hover{
  background:#0056b3;
}
```

(d) Performance and User Experience Considerations

Performance

- Use transform and opacity for smooth animations because they are GPU-accelerated.
- Avoid animating expensive properties like width and height.
- Keep animation duration between **0.2–0.4 seconds** for responsiveness.

User Experience

- Hover effects should provide visual feedback without distracting users.
- Ensure buttons remain easy to click.
- Maintain readable text and sufficient color contrast.
- Keep animations subtle to improve the shopping experience.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).
- (b) Compare **Flexbox and Grid** based on layout needs.
- (c) Choose the most suitable layout technique and justify your choice.
- (d) Evaluate the scalability and responsiveness of the chosen approach.

(a) Layout requirements of a dashboard

A web application dashboard typically requires:

- **Rows and columns** for organizing content.
- **Header** for logo and navigation.
- **Sidebar** for menu options.
- **Main content area** for charts, tables, and reports.
- **Cards/widgets** aligned properly with equal spacing.
- **Responsive layout** that adapts to different screen sizes.
-

(b) Comparison of Flexbox and Grid

Feature	Flexbox	Grid
Layout Type	One-dimensional (Row or Column)	Two-dimensional (Rows and Columns)
Best For	Navigation bars, menus, small components	Complete page layouts, dashboards
Alignment	Excellent	Excellent
Complex Layouts	Limited	Very Easy
Responsiveness	Good	Excellent

(c) Most suitable layout technique

CSS Grid is the most suitable choice for a dashboard because:

- It supports **both rows and columns**.
- Easily creates **header, sidebar, content, and footer** layouts.
- Reduces the amount of CSS required.
- Provides better control over spacing and alignment.
- Makes complex dashboard layouts easier to maintain.

Example:

```
.container{  
  display:grid;  
  grid-template-columns:200px 1fr;  
  grid-template-rows:70px auto;  
  gap:10px;  
}
```


(d) Scalability and Responsiveness

Scalability

- New widgets and dashboard sections can be added without changing the entire layout.
- Grid automatically organizes content efficiently.

Responsiveness

- Media queries can rearrange the layout for tablets and mobile devices.
- Columns can stack vertically on smaller screens.
- Ensures a consistent and user-friendly interface across devices.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- Identify key sections of a blog layout.
- Design a mobile-first layout using CSS.
- Demonstrate how the layout scales for larger screens using media queries.
- Evaluate accessibility and readability of the design.

(a) Key sections of a blog layout

A mobile-first blog layout should include:

- **Header** – Blog title/logo and navigation.
- **Main Content** – Blog articles with images and text.
- **Sidebar** (*optional on desktop*) – Categories, recent posts, archives.
- **Footer** – Copyright, social media links, contact information.
- **Responsive Navigation** – Easy access to menu on mobile devices.

(b) Mobile-First Layout using CSS

HTML

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="style.css">
</head>
<body>

<header>My Blog</header>

<main>
  <article>
    <h2>Welcome to My Blog</h2>
    <p>This is a sample blog post.</p>
  </article>
</main>

<footer>
  © 2026 My Blog
</footer>

</body>
</html>

CSS
body{
  margin:0;
  font-family:Arial,sans-serif;
}

header,footer{
  background:#333;
  color:white;
```

```
    text-align:center;
    padding:15px;
}

main{
    padding:20px;
}

article{
    background:#f4f4f4;
    padding:15px;
    border-radius:8px;
}
```

(c) Layout scaling using Media Queries

```
@media(min-width:768px){

main{
    display:flex;
    gap:20px;
}

article{
    flex:3;
}

aside{
    flex:1;
}
}
```

Explanation:

- **Mobile (default):** Content is displayed in a single column.
- **Tablet/Desktop (768px and above):** Content changes to multiple columns with the article and sidebar displayed side by side.

(d) Accessibility and Readability

Accessibility

- Use semantic HTML elements (header, main, article, footer).
- Maintain sufficient color contrast.
- Ensure keyboard navigation is supported.
- Add alt text for images.

Readability

- Use clear fonts (e.g., Arial).
- Maintain proper spacing and margins.
- Use readable font sizes (16px or larger).
- Keep line spacing comfortable for easy reading.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

(a) Appropriate JavaScript events for menu interaction

Common JavaScript events used for a dynamic menu are:

- **click** – Opens or closes the menu.
- **mouseover** – Displays submenu when the mouse hovers.
- **mouseout** – Hides submenu when the mouse leaves.
- **mouseenter** – Activates menu on hover without repeated triggering.
- **mouseleave** – Closes the menu when the cursor exits.

(b) Event handling logic for menu toggle functionality

Logic:

1. Select the menu button and navigation menu.
2. Listen for the **click** event.
3. Toggle a CSS class (e.g., show) to display or hide the menu.

(c) Implementation for user actions (Click/Hover)

HTML

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="style.css">
</head>
<body>

<button id="menuBtn">☰ Menu</button>

<ul id="menu">
  <li>Home</li>
  <li>Products</li>
  <li>About</li>
  <li>Contact</li>
</ul>

<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

CSS

```
#menu{  
  display:none;  
  list-style:none;  
  background:#333;  
  color:white;  
  padding:10px;  
}
```

```
.show{  
  display:block;  
}
```

```
li{  
  padding:10px;  
  cursor:pointer;  
}
```

```
li:hover{  
  background:#555;  
}
```

JavaScript

```
const btn = document.getElementById("menuBtn");  
const menu = document.getElementById("menu");
```

```
btn.addEventListener("click", function(){  
  menu.classList.toggle("show");  
});
```

(d) Performance and Suggested Improvements

Performance

- Use `classList.toggle()` instead of repeatedly changing styles.
- Avoid attaching unnecessary event listeners.
- Keep JavaScript code simple and lightweight.

Suggested Improvements

- Add smooth open/close animations using CSS transitions.
- Support keyboard navigation for accessibility.
- Close the menu automatically when clicking outside it.
- Add ARIA attributes (`aria-expanded`, `aria-controls`) for better accessibility.
- Optimize for touch devices to improve the mobile experience.